

ANALISE TECNICA COMPLETA

bundle.social - Unified Social Media API

Site analisado	https://bundle.social
API base URL	https://api.bundle.social
Docs	https://docs.bundle.social
OpenAPI spec	https://api.bundle.social/swagger-json
SDK GitHub	github.com/bundleglobal/bundlesocial-node
Status page	bundlesocial.betteruptime.com
Data da analise	Novembro 2025
Objetivo	Engenharia reversa + blueprint para clone superior

Resumo executivo: bundle.social e uma API unificada para postar, agendar, analisar e moderar conteudo em 15 redes sociais atraves de uma unica integracao. O modelo de negocio cobra por organizacao (nao por conta conectada), com planos gratuito, Pro (\$100/mes), Business (\$400/mes) e Enterprise. A arquitetura e multi-tenant por design, com hierarquia Organizacao > Team > Social Accounts. Este documento mapeia toda a arquitetura, endpoints, autenticacao, webhooks, rate limits, modelo de dados e tecnologias utilizadas, fornecendo um blueprint para construir uma alternativa superior.

1. VISAO GERAL DO PRODUTO

1.1 O que e bundle.social

bundle.social e uma camada de abstracao (middleware) que unifica 15 APIs de redes sociais diferentes em uma unica API REST. Em vez de integrar Facebook, Instagram, TikTok, LinkedIn, X, YouTube, etc. separadamente - cada uma com seu proprio fluxo OAuth, estrutura de payload, regras de publicacao e modos de falha - o desenvolvedor integra uma vez com bundle.social e publica em todas as plataformas.

1.2 Plataformas suportadas (15)

Plataforma	OAuth?	set-channel?	Comentarios?	Analytics?
Facebook	Nao	Sim (Page)	Sim	Sim
Instagram (FB)	Nao	Sim (conta IG)	Sim	Sim
Instagram (direto)	Sim	Nao	Sim	Sim
X / Twitter	Sim	Nao	Nao	Nao
TikTok	Sim	Nao	Sim	Sim
LinkedIn	Nao	Sim (perfil/Page)	Sim	Sim
YouTube	Nao	Sim (canal)	Sim	Sim
Pinterest	Sim	Nao (boards refresh)	Nao	Sim
Reddit	Sim	Nao	Sim	Limitado
Discord	Sim (webhook)	Nao	Sim	Nao
Slack	Sim (webhook)	Nao	Sim	Nao
Mastodon	Sim	Nao	Sim	Basico
Threads	Sim	Nao	Sim	Sim
Bluesky	Sim	Nao	Sim	Calculado
Google Business	Nao	Sim (local)	Nao	Sim
Snapchat	Sim	Nao	Nao	Sim

Insight de design: A coluna 'set-channel' revela que Meta (Facebook/Instagram), LinkedIn, YouTube e Google Business exigem um segundo passo apos OAuth: selecionar o alvo exato (Page, perfil, canal, local). Isso e uma abstracao chave - bundle.social separa 'OAuth (usuario concedeu acesso)' de 'channel selection (qual alvo usar)'.

1.3 Modelo de negocio

Plano	Preco	Posts/mes	Comentarios/mes	Contas sociais	Destaque
FREE	\$0	20	50	3	Teste
PRO	\$100/mes	10.000	5.000	Ilimitado	Power users
BUSINESS	\$400/mes	100.000	50.000	Ilimitado	Empresas
CUSTOM	Sob consulta	Custom	Custom	Ilimitado	Enterprise

Modelo chave: Cobranca por ORGANIZACAO, nao por conta conectada. 3 contas = 3.000 contas no mesmo preco. A diferenciacao e o volume mensal de posts. Isso e ideal para SaaS multi-tenant e agencias.

2. ARQUITETURA TECNICA

2.1 Hierarquia de dados (Multi-tenant)

A arquitetura e multi-tenant com hierarquia Organizacao > Team > Social Accounts. Quase toda chamada de API exige um teamId. Essa e a estrutura central:

```
Organization (sua conta) |- Team A (ex: "Marketing") | |- Social Accounts (Instagram, TikTok, ...) | |- Posts | |- Uploads |- Team B (ex: "Cliente: Acme Corp") | |- Social Accounts | |- Posts | |- Uploads |- API Keys (compartilhadas entre todos os teams) |- Webhooks (compartilhados entre todos os teams) |- Subscription & Billing
```

2.2 O que mora em cada nivel

Nivel	O que pertence
Organization	API keys, webhooks, subscription/billing, configuracoes de compressao
Team	Social accounts, posts, uploads, bots, convites, bio/link-in-bio

Regra critica: Rate limits sao por TEAM. Post caps mensais sao por ORGANIZATION. Para SaaS onde cada cliente precisa suas proprias contas, crie um TEAM por cliente - eles nao pisarao uns nos outros.

2.3 Stack tecnologico inferido

Componente	Tecnologia (inferida)	Evidencia
Backend API	Node.js + Express/NestJS	SDK em TypeScript, OpenAPI 3.0, padrao REST
Linguagem SDK	TypeScript	package.json: typescript ^5.4.5, tsup build
Geracao de cliente	@hey-api/openapi-ts	package.json devDeps
Testes SDK	Jest	jest.config.ts, ts-jest
Release SDK	semantic-release	.releaserc, @semantic-release/git
Documentacao	Mintlify / custom	Padrao de cards, llms.txt, info.bundle.social
Storage de midia	S3-compativel (presigned URLs)	Multipart upload com presigned PUT URLs
Banco de dados	PostgreSQL (provavel)	Padrao de IDs (post_abc, team_xyz, sa_789)
Cache/Rate limit	Redis (provavel)	Rate limiting em 3 camadas, contador por endpoint
Webhooks delivery	Fila assincrona (BullMQ/SQS)	Retries exponenciais, 50 concorrentes
Status page	BetterStack (Better Uptime)	Subdominio bundlesocial.betteruptime.com
Frontend dashboard	React/Next.js (provavel)	Padrao SPA, dark mode toggle, dashboard
Autenticacao	API Key (x-api-key)	Header unico, sem OAuth2 para clientes
Spec API	OpenAPI 3.0	swagger-json, swagger-yaml publicos
MCP Server	Model Context Protocol	bundlesocial-mcp para Claude/Cursor
CLI	Node.js CLI	bundlesocial-cli, JSON in/out

3. AUTENTICACAO E SEGURANCA

3.1 API Key (cliente -> bundle.social)

Autenticacao simples via header x-api-key. Sem OAuth2, sem JWT para o cliente final. A chave tem prefixo pk_live_... (padrao Stripe-like).

```
GET /api/v1/post Host: api.bundle.social x-api-key: pk_live_XXXXXXXXXXXX Content-Type: application/json
```

401 Unauthorized = sem chave. **403 Forbidden** = chave invalida. Chaves sao org-level: uma chave de acesso a todos os teams da organizacao.

3.2 OAuth (bundle.social -> plataformas sociais)

bundle.social gerencia TODO o fluxo OAuth com cada plataforma. O desenvolvedor que consome a API nunca toca nos tokens das redes sociais. Existem dois fluxos:

Fluxo Hosted (recomendado)

Voce cria um 'connect link' via API. bundle.social abre um portal que gerencia OAuth UI, selecao de canal (Page/canal/local), idioma e tudo mais. O usuario conclui e e redirecionado de volta.

Fluxo Custom UI

Voce constroi sua propria UI de conexao. Chama o endpoint /connect, gerencia o callback OAuth, e usa /set-channel para selecionar o alvo. Mais controle, mais trabalho.

3.3 Token management

bundle.social armazena accessToken, refreshToken, secret e expiresAt de cada conta social. Esses campos NUNCA aparecem em webhooks ou respostas publicas. Renovacao automatica de tokens. Deteccao de desconexao remota a cada 6 horas (Meta: Facebook, Instagram, Threads).

3.4 Webhook signature verification

Cada webhook vem com header x-signature contendo HMAC-SHA256. Voce verifica com seu Signing Secret:

```
import crypto from 'crypto'; function verifyWebhook(rawBody, signature, secret) { const expected = crypto.createHmac('sha256', secret).update(rawBody).digest('hex'); return crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(signature)); }
```

4. ENDPOINTS DA API (mapeados)

Base URL: <https://api.bundle.social> - Tudo prefixado com /api/v1

4.1 Posts

Endpoint	Metodo	Funcao
/api/v1/post	POST	Criar post (rascunho, agendado ou imediato)
/api/v1/post	GET	Listar posts de um team
/api/v1/post/:id	GET	Detalhes de um post
/api/v1/post/:id	PATCH	Atualizar post (antes de publicar)
/api/v1/post/:id	DELETE	Deletar post
/api/v1/post/bulk-csv	POST	Criar varios posts via CSV (async)

4.2 Uploads (Midia)

Endpoint	Metodo	Funcao
/api/v1/upload	POST	Simple upload (multipart/form-data, ate 90MB)
/api/v1/upload/init	POST	Direct upload - retorna presigned URL (ate 5GiB)
/api/v1/upload/finalize	POST	Finaliza direct upload
/api/v1/upload/multipart/init	POST	Inicia multipart (retorna URLs por parte)
/api/v1/upload/multipart/sign	POST	Re-assina partes expiradas
/api/v1/upload/multipart/complete	POST	Junta partes e registra upload
/api/v1/upload/multipart/abort	POST	Cancela multipart upload
/api/v1/upload/from-url	POST	Registra midia de URL publica (ate 1GB)

Estrategia de upload: 3 metodos. Simple (imagens, 90MB), Direct (presigned PUT, 5GiB), Multipart (videos grandes, chunks de 64MiB, retryable). Multipart e o unico que sobrevive a queda de rede - cada chunk e independente. URLs presigned expiram em 6h (multipart) ou 30min (direct).

4.3 Social Accounts

Endpoint	Metodo	Funcao
/api/v1/social-account/connect	POST	Iniciar fluxo OAuth (custom UI)
/api/v1/social-account/connect-link	POST	Criar hosted connect link
/api/v1/social-account/set-channel	POST	Selecionar canal (Page, canal, local)
/api/v1/social-account/refresh-channels	POST	Atualizar lista de canais
/api/v1/social-account/unset-channel	POST	Remover canal selecionado
/api/v1/social-account/connection-check	POST	Validar token manualmente
/api/v1/social-account	GET	Listar contas de um team
/api/v1/social-account/:id	DELETE	Desconectar conta

4.4 Analytics

Endpoint	Metodo	Funcao
/api/v1/analytics/social-account	GET	Analytics de conta (views, likes, demographics)
/api/v1/analytics/social-account/raw	GET	Payload bruto por plataforma
/api/v1/analytics/social-account/force	POST	Forcar refresh (rate limited)
/api/v1/analytics/post	GET	Analytics de post
/api/v1/analytics/post/raw	GET	Payload bruto do post
/api/v1/analytics/post/force	POST	Forcar refresh de post
/api/v1/analytics/post/bulk	GET	Analytics em massa de posts de um team

4.5 Comments

Endpoint	Metodo	Funcao
/api/v1/comment	POST	Criar comentario em post publicado
/api/v1/comment	GET	Listar comentarios importados
/api/v1/comment/import	POST	Importar comentarios de um post

4.6 Organization, Teams, Webhooks, Health

Endpoint	Metodo	Funcao
/api/v1	GET	Health check + status por plataforma
/api/v1/organization	GET	Dados da organizacao
/api/v1/organization/usage/daily-limits	GET	Uso diario por conta
/api/v1/team	POST/GET	Criar/listar teams
/api/v1/team/:id	PATCH/DELETE	Atualizar/deletar team
/api/v1/webhook	POST/GET	Configurar webhooks (org-level)
/api/v1/import/posts	POST	Importar historico de posts (async job)

5. ESTRUTURA DE UM POST

O post é o objeto central. A genialidade do design está no campo 'data' - um objeto chaveado por plataforma. Cada plataforma recebe seus próprios campos.

```
POST /api/v1/post { "teamId": "team_xyz", "title": "Meu post", "status": "SCHEDULED", //  
DRAFT | SCHEDULED | POSTED | ERROR "postDate": "2026-01-15T10:00:00.000Z",  
"socialAccountTypes": ["TIKTOK", "YOUTUBE"], "data": { "TIKTOK": { "text": "Video viral",  
"uploadIds": ["upload_456"], "privacy": "PUBLIC_TO_EVERYONE" }, "YOUTUBE": { "text":  
"Shorts cat", "type": "SHORT", "madeForKids": false, "uploadIds": ["upload_456"] } } }
```

5.1 Campos por plataforma (data.PLATFORM)

X / Twitter (data.TWITTER)

text (até 280 chars), uploadIds (até 4 imagens OU 1 vídeo), replySettings, isAiGenerated. Sem analytics.

Instagram (data.INSTAGRAM)

text, uploadIds, type (POST/REEL/STORY), aspect ratio 4:5 a 1.91:1. Reels suportam música.

TikTok (data.TIKTOK)

text, uploadIds, privacy (PUBLIC_TO_EVERYONE, etc), música via CML (Commercial Music Library).

YouTube (data.YOUTUBE)

text, uploadIds, type (SHORT ou VIDEO), madeForKids (obrigatório), title (sempre obrigatório). Vídeos até 4h e 5GB.

LinkedIn (data.LINKEDIN)

text (até 3000 chars, obrigatório), uploadIds, visibility (PUBLIC/CONNECTIONS). URNs para Company Pages.

Threads (data.THREADS)

text (até 500 chars), uploadIds (até 10 imagens), topicTag, replyControl, poll, gif, linkAttachment, crosspostToInstagramStory.

Bluesky (data.BLUESKY)

text (até 300 chars), uploadIds (4 imgs ou 1 vídeo), tags, labels (content warnings), quoteUri, externalUrl.

Mastodon (data.MASTODON)

text (depende da instância), uploadIds, privacy (PUBLIC/UNLISTED/PRIVATE/DIRECT), spoiler.

Facebook (data.FACEBOOK)

text, uploadIds, type (POST/REEL/STORY). Token expiration é a dor principal.

6. WEBHOOKS E EVENTOS

6.1 Eventos suportados

Evento	Quando dispara
post.published	Post terminou processamento (POSTED ou ERROR)
comment.published	Comentario terminou (POSTED ou ERROR)
social-account.created	Usuario conectou nova conta social
social-account.updated	Conta mudou (canal removido, desconexao remota)
social-account.deleted	Conta foi desconectada/removida
team.created	Novo team criado
team.updated	Detalhes do team mudaram (inclui add/remove de contas)
team.deleted	Team deletado

6.2 Estrutura do payload

```
{ "type": "post.published", "data": { "id": "post_abc123", "status": "POSTED", // ou "ERROR" "postDate": "2026-01-15T10:00:00.000Z", "postedDate": "2026-01-15T10:00:02.341Z", "teamId": "team_xyz", "organizationId": "org_123", "data": { "INSTAGRAM": { ... } }, "error": null, "errorsVerbose": null, "externalData": { "INSTAGRAM": { "id": "17900000000000000", "permalink": "https://instagram.com/p/example/" } }, "retryCount": 0, "uploads": [...], "socialAccounts": [...] } }
```

6.3 Headers enviados

Header	Valor
Content-Type	application/json
User-Agent	bundlesocial
x-signature	HMAC-SHA256 (verificar com Signing Secret)

6.4 Delivery e retries

Configuracao	Valor
Timeout	15 segundos por request
Max tentativas	3 (inicial + 2 retries)
Backoff	Exponencial, comecando em 30s
Concorrencia	Ate 50 deliveries simultaneos
Auto-disable	7 dias sem sucesso = webhook desativado
Re-envio	Manual via dashboard ou API

7. RATE LIMITS E COTAS

7.1 Rate limits da API (3 camadas)

Camada	Janela	Max Requests	Equivalente	Previne
Burst	1 segundo	100	6.000/min	Promise.all() com 500 reqs
Short	10 segundos	500	3.000/min	Polling agressivo/loops
Minute	1 minuto	2.000	2.000/min	Abuso sustentado

Implementacao: Contadores por endpoint + por tracker (API key, bearer token, ou IP). Trafego pesado em POST /post nao consome o bucket de GET /analytics. Resposta 429 quando excedido.

7.2 Limites diarios por plataforma (por conta conectada)

Plataforma	FREE	PRO	BUSINESS
Twitter/X	5	15	15
Facebook	10	50	100
Instagram	10	50	100
LinkedIn	10	18	24
YouTube	10	10	15
TikTok	5	10	15
Threads	10	200	250
Pinterest	10	24	36
Reddit	10	24	36
Discord	10	100	200
Slack	10	100	200
Mastodon	10	50	100
Bluesky	10	50	100
Google Business	10	20	40
Snapchat	5	20	40

Regra chave: A cota segue a CONTA REAL (platform + account id), nao a conexao. A mesma conta TikTok conectada a 5 teams = 10/day total (nao 50). Contas diferentes = cotas independentes.

7.3 Limites mensais (organizacao)

Tier	Posts/mes	Comentarios/mes	Uploads/mes
FREE	20	50	200
PRO	10.000	5.000	100.000
BUSINESS	100.000	50.000	1.000.000

Contagem: Por data de CRIACAO (quando voce agenda), no mes UTC. Agendar para mes futuro consome cota do mes ATUAL. Rascunhos nao contam. Falhas e delecoes NAO devolvem cota. Reset no dia 1 de cada mes (UTC).

7.4 Force refresh analytics

Formula: max force refreshes/dia = numero_de_teams x 5. Ex: 10 teams = 50 refreshes/dia. Exceder = 429.

8. SISTEMA DE ERROS PADRONIZADO

Erros são normalizados entre todas as plataformas. Cada erro tem code estável, userFacingMessage (seguro pra usuário), errorMessage (log dev), e isTransient (retry vs desistir).

8.1 Estrutura

```
{ "errorsVerbose": { "INSTAGRAM": { "code": "META:190", "userFacingMessage": "Seu token do Instagram expirou. Reconecte.", "errorMessage": "OAuthException: Validate permission failed.", "isTransient": false }, "TIKTOK": null // Sucesso! } }
```

8.2 Prefixos por plataforma

Prefixo	Plataforma
META	Instagram, Facebook, Threads
TT	TikTok
LI	LinkedIn
YT	YouTube
HTTP	Erros genéricos de API

8.3 Transient vs Non-Transient

Transient (true): Rate limits, timeouts. Ação: retry com backoff exponencial.

Non-Transient (false): Auth errors, validação. Ação: pedir usuário para corrigir (reconectar conta, mudar arquivo).

8.4 Erros canônicos mais comuns

Error key	Plataforma	Causa
account_connection_expired	Facebook	Token/sessão invalidado
page_or_target_inaccessible	Meta	Página não acessível
media_fetch_failed	Meta	Crawler bloqueado (403)
account_authorization_revoked	Meta	App authorization removida
posting_rate_limited	Meta	Limite de frequência/spam
identity_confirmation_required	Meta	Confirmar identidade no app
missing_platform_permission	Meta	Permissões de Page revogadas
account_security_limited	Meta	Conta limitada por segurança
platform_login_required	Meta	Checkpoint de login

9. DATA RETENTION (Politica de retencao)

Tipo de dado	Retencao
Analytics (parsed + raw)	30 dias
Webhook events	7 dias
Uploads deletados	7 dias apos delecao
Uploads nao usados	Soft-delete apos 90 dias, + 7 dias
Posts importados	30 dias

Regra de ouro: Se voce precisa de historico longo, armazene voce mesmo. bundle.social e otimizado para publicacao em tempo real e analytics recentes, nao para armazenamento permanente. Cron job diario que busca analytics e salva no seu banco e a pratica recomendada.

10. BLUEPRINT PARA CONSTRUIR ALTERNATIVA SUPERIOR

Abaixo, o plano tecnico para construir uma alternativa ao bundle.social. Cada secao indica o que eles fizeram e como fazer MELHOR.

10.1 Stack recomendada (superior)

Camada	bundle.social (inferido)	Recomendado (superior)
Backend	Node.js + Express/Nest	Node.js + NestJS + Fastify (mais rapido)
Linguagem	TypeScript	TypeScript (strict mode)
Banco principal	PostgreSQL (provavel)	PostgreSQL 16 + Prisma/Drizzle ORM
Cache/Rate limit	Redis (provavel)	Redis 7 + Upstash (serverless)
Fila de jobs	BullMQ/SQS (inferido)	BullMQ + Redis (ou Temporal para workflows)
Storage midia	S3-compativel	Cloudflare R2 (sem egress) ou MinIO self-host
Frontend	React/Next.js (provavel)	Next.js 15 + App Router + Server Components
Auth	API Key (x-api-key)	API Key + JWT opcional + RBAC granular
Docs	Mintlify/custom	Mintlify ou Fuma (com llms.txt)
Spec	OpenAPI 3.0	OpenAPI 3.1 + Zod schemas
SDK	TypeScript (openapi-ts)	TS + Python + Go (gerar de OpenAPI)
Monitoring	BetterStack	Grafana + Loki + Prometheus (self-host)
Deploy	Desconhecido	Docker + Coolify ou Kubernetes
CDN	Cloudflare (inferido)	Cloudflare (free tier)
MCP server	Sim	Sim + A2A (Agent-to-Agent) protocol

10.2 Modelo de dados (PostgreSQL)

```
-- Hierarquia: Organization > Team > SocialAccount
CREATE TABLE organizations ( id TEXT PRIMARY KEY DEFAULT 'org_' || gen_random_uuid(), name TEXT NOT NULL, plan TEXT DEFAULT 'FREE', subscription_status TEXT, created_at TIMESTAMPTZ DEFAULT NOW() );
CREATE TABLE teams ( id TEXT PRIMARY KEY DEFAULT 'team_' || gen_random_uuid(), organization_id TEXT REFERENCES organizations(id) ON DELETE CASCADE, name TEXT NOT NULL, avatar_url TEXT, created_at TIMESTAMPTZ DEFAULT NOW() );
CREATE TABLE api_keys ( id TEXT PRIMARY KEY DEFAULT 'key_' || gen_random_uuid(), organization_id TEXT REFERENCES organizations(id) ON DELETE CASCADE, key_hash TEXT NOT NULL, -- bcrypt/sha256 do pk_live... prefix TEXT, -- pk_live_xxxx (para identificacao) last_used_at TIMESTAMPTZ, created_at TIMESTAMPTZ DEFAULT NOW() );
CREATE TABLE social_accounts ( id TEXT PRIMARY KEY DEFAULT 'sa_' || gen_random_uuid(), team_id TEXT REFERENCES teams(id) ON DELETE CASCADE, type TEXT NOT NULL, -- INSTAGRAM, TIKTOK, etc username TEXT, display_name TEXT, external_id TEXT, -- ID na plataforma access_token TEXT, -- encryptado! refresh_token TEXT, -- encryptado! token_expires_at TIMESTAMPTZ, channel_id TEXT, -- Page/canal selecionado channel_data JSONB, -- metadata do canal status TEXT DEFAULT 'ACTIVE', created_at TIMESTAMPTZ DEFAULT NOW() );
CREATE TABLE posts ( id TEXT PRIMARY KEY DEFAULT 'post_' || gen_random_uuid(), team_id TEXT REFERENCES teams(id) ON DELETE CASCADE, title TEXT, status TEXT DEFAULT 'DRAFT', -- DRAFT|SCHEDULED|POSTED|ERROR post_date TIMESTAMPTZ, posted_date TIMESTAMPTZ, data JSONB NOT NULL, -- { INSTAGRAM: {...}, TIKTOK: {...} } external_data JSONB, -- IDs/links retornados por plataforma error TEXT, errors_verbose JSONB, retry_count INT DEFAULT 0, created_at TIMESTAMPTZ DEFAULT NOW(), updated_at TIMESTAMPTZ DEFAULT NOW() );
CREATE TABLE uploads ( id TEXT PRIMARY KEY DEFAULT 'upload_' || gen_random_uuid(),
```

```
team_id TEXT REFERENCES teams(id) ON DELETE CASCADE, file_name TEXT NOT NULL, mime_type
TEXT NOT NULL, size BIGINT, storage_path TEXT NOT NULL, -- path no S3/R2 status TEXT
DEFAULT 'ACTIVE', -- ACTIVE|DELETED deleted_at TIMESTAMPTZ, created_at TIMESTAMPTZ
DEFAULT NOW() ); CREATE TABLE webhooks ( id TEXT PRIMARY KEY DEFAULT 'wh_' ||
gen_random_uuid(), organization_id TEXT REFERENCES organizations(id) ON DELETE CASCADE,
url TEXT NOT NULL, signing_secret TEXT NOT NULL, status TEXT DEFAULT 'ACTIVE', --
ACTIVE|DISABLED last_success_at TIMESTAMPTZ, failure_count INT DEFAULT 0, created_at
TIMESTAMPTZ DEFAULT NOW() ); CREATE TABLE webhook_events ( id TEXT PRIMARY KEY DEFAULT
'ev_' || gen_random_uuid(), webhook_id TEXT REFERENCES webhooks(id) ON DELETE CASCADE,
type TEXT NOT NULL, -- post.published, etc payload JSONB NOT NULL, status TEXT, --
PENDING|DELIVERED|FAILED response_code INT, attempts INT DEFAULT 0, next_retry_at
TIMESTAMPTZ, created_at TIMESTAMPTZ DEFAULT NOW() ); CREATE TABLE analytics_snapshots (
id TEXT PRIMARY KEY DEFAULT 'an_' || gen_random_uuid(), social_account_id TEXT REFERENCES
social_accounts(id) ON DELETE CASCADE, post_id TEXT REFERENCES posts(id) ON DELETE
CASCADE, platform_type TEXT NOT NULL, metrics JSONB NOT NULL, -- views, likes,
impressions, demographics raw JSONB, -- payload bruto da plataforma forced BOOLEAN
DEFAULT FALSE, fetched_at TIMESTAMPTZ DEFAULT NOW() ); CREATE TABLE usage_counters ( id
TEXT PRIMARY KEY DEFAULT 'uc_' || gen_random_uuid(), organization_id TEXT REFERENCES
organizations(id), team_id TEXT REFERENCES teams(id), counter_type TEXT NOT NULL, --
posts|comments|uploads|imports period TEXT NOT NULL, -- 2026-01 (mes UTC) ou 2026-01-15
(dia) used INT DEFAULT 0, UNIQUE(team_id, counter_type, period) );
```

10.3 Arquitetura de roteamento (publishing pipeline)

O coracao do produto e o pipeline de publicacao. Um post pode ir para N plataformas simultaneamente, cada uma com seu proprio fluxo, timeout e modo de falha.

```
[Cliente] POST /api/v1/post | v [API Gateway] -- valida API key, rate limit (3 camadas) |
v [Post Service] -- salva no DB como SCHEDULED | v [Scheduler (BullMQ)] -- agenda job
para postDate | v (quando postDate chega) [Publisher Worker] -- para cada plataforma em
data: | +-- Instagram Publisher -> API Meta -> sucesso/erro +-- TikTok Publisher -> API
TikTok -> sucesso/erro +-- YouTube Publisher -> API YouTube -> sucesso/erro +-- ...
(paralelo, cada um independente) | v [Resultado agregado] | +-- atualiza post.status
(POSTED se todos OK, ERROR se algum falhou) +-- salva externalData (IDs/permalinks
retornados) +-- dispara webhook post.published +-- se erro transient -> agenda retry +--
se erro non-transient -> marca ERROR, notifica
```

10.4 Fluxo OAuth por plataforma

Cada plataforma tem seu proprio adapter OAuth. O design correto e o padrao Adapter/Strategy:

```
interface PlatformAdapter { getAuthUrl(state: string, options?: OAuthOptions): string;
handleCallback(code: string): Promise<TokenSet>; refreshToken(refreshToken: string):
Promise<TokenSet>; getChannels(tokenSet: TokenSet): Promise<Channel[]>; publish(post:
PostData, account: SocialAccount): Promise<PublishResult>; getAnalytics(account:
SocialAccount, postId?: string): Promise<Analytics>; validateConnection(account:
SocialAccount): Promise<boolean>; } // Implementacoes: // - InstagramAdapter (via Meta
Graph API) // - FacebookAdapter (via Meta Graph API, Page token) // - TikTokAdapter (via
TikTok Content API) // - YouTubeAdapter (via YouTube Data API v3) // - LinkedInAdapter
(via LinkedIn Marketing API) // - TwitterAdapter (via X API v2) // - ThreadsAdapter (via
Threads API) // - BlueskyAdapter (via AT Protocol) // - MastodonAdapter (via Mastodon
API, instancia custom) // - PinterestAdapter (via Pinterest API v5) // - RedditAdapter
(via Reddit API) // - DiscordAdapter (via webhook URL) // - SlackAdapter (via webhook
URL) // - GoogleBusinessAdapter (via Google My Business API) // - SnapchatAdapter (via
Snapchat Marketing API)
```

10.5 Sistema de rate limiting (3 camadas)

```
// Middleware Fastify/Express async function rateLimiter(req, reply, done) { const
tracker = req.headers['x-api-key'] || req.ip; const endpoint =
`${req.method}:${req.routeOptions.url}`; const now = Date.now(); // 3 janelas: 1s (burst:
100), 10s (short: 500), 60s (minute: 2000) const windows = [ { key:
`${tracker}:${endpoint}:1s`, limit: 100, ttl: 1 }, { key: `${tracker}:${endpoint}:10s`,
limit: 500, ttl: 10 }, { key: `${tracker}:${endpoint}:60s`, limit: 2000, ttl: 60 }, ];
for (const w of windows) { const count = await redis.incr(w.key); if (count === 1) await
redis.expire(w.key, w.ttl); if (count > w.limit) { return reply.code(429).send({ error:
'RATE_LIMITED', window: w.ttl + 's', limit: w.limit }); } } done(); }
```

10.6 Upload de midia (3 estrategias)

```
// 1. SIMPLE (imagens, <90MB) - multipart/form-data direto POST /api/v1/upload form-data:
file, teamId -> salva no R2/S3, retorna upload.id // 2. DIRECT (ate 5GiB) - presigned URL
POST /api/v1/upload/init -> { url, path } PUT <url> --upload-file video.mp4 // direto no
storage POST /api/v1/upload/finalize -> { id } // 3. MULTIPART (videos grandes,
retryable) - chunks de 64MiB POST /api/v1/upload/multipart/init -> { uploadId, parts:
[<url>] } PUT <part1.url> --upload-file chunk1 // cada parte independente PUT <part2.url>
--upload-file chunk2 POST /api/v1/upload/multipart/complete -> { id } // Se uma parte
falhar, re-assina apenas ela: POST /api/v1/upload/multipart/sign { partNumbers: [3] }
```

10.7 Webhook delivery system

```
// Worker que processa a fila de webhooks async function webhookWorker(event:
WebhookEvent) { const webhook = await getWebhook(event.webhookId); if (webhook.status ===
'DISABLED') return; const payload = JSON.stringify({ type: event.type, data:
event.payload }); const signature = hmacSHA256(payload, webhook.signing_secret); try {
const res = await fetch(webhook.url, { method: 'POST', headers: { 'Content-Type':
'application/json', 'User-Agent': 'yourapp', 'x-signature': signature }, body: payload,
signal: AbortSignal.timeout(15000) // 15s timeout }); if (res.ok) { await
markDelivered(event.id); await resetFailureCount(webhook.id); } else { throw new
Error(`HTTP ${res.status}`); } } catch (err) { await incrementAttempts(event.id); if
(event.attempts < 3) { // Backoff exponencial: 30s, 90s const delay = 30 * Math.pow(3,
event.attempts - 1); await scheduleRetry(event.id, delay); } else { await
markFailed(event.id); await incrementFailureCount(webhook.id); // Auto-disable apos 7
dias sem sucesso if (daysSinceLastSuccess(webhook.id) >= 7) { await
disableWebhook(webhook.id); await notifyOwner(webhook.organizationId); } } } }
```

10.8 Cron jobs necessarios

Job	Frequencia	Funcao
Analytics refresh	A cada 24h	Buscar analytics de todas as contas
Connection check (Meta)	A cada 6h	Validar tokens Meta (FB, IG, Threads)
Token refresh	Proativo	Renovar tokens prestes a expirar
Stale upload cleanup	Diario	Soft-delete uploads nao usados ha 90 dias
Deleted upload purge	Diario	Remover fisicamente uploads deletados ha 7 dias
Analytics purge	Diario	Deletar analytics com 30 dias
Webhook events purge	Diario	Deletar eventos com 7 dias
Imported posts purge	Diario	Deletar posts importados com 30 dias
Monthly counter reset	Dia 1 (UTC)	Zerar contadores mensais de uso
Webhook auto-disable check	Diario	Desativar webhooks sem sucesso ha 7 dias

11. MELHORIAS PARA SUPERAR BUNDLE.SOCIAL

Area	bundle.social	Sua versao (superior)
Retencao analytics	30 dias (limite duro)	Configuravel: 30/90/365/infinito (tier-based)
Webhook retries	3 tentativas, 30s backoff	5 tentativas, backoff configuravel + dead-letter queue
Webhook auto-disable	7 dias sem sucesso	Configuravel + alertas antes de desativar
SDKs	TypeScript apenas	TS + Python + Go + PHP + Ruby (gerar de OpenAPI)
MCP	Sim (Claude/Cursor)	MCP + A2A + OpenAI function calling nativo
Rate limit headers	Nao documentado	X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset
Dashboard	Web only	Web + mobile app (React Native)
Analytics historico	Cliente que se vire	Data warehouse integrado (ClickHouse)
Multi-idioma	Ingles	PT-BR, EN, ES nativo no portal OAuth
White-label	Menciona mas nao detalha	Full white-label: dominio proprio, logo, cores
Storage	S3 (com egress)	Cloudflare R2 (zero egress) ou self-host MinIO
Observabilidade	BetterStack externo	Grafana + Loki + Prometheus integrados
Pricing	4 tiers fixos	4 tiers + pay-as-you-go + self-hosted option
Compliance	Nao mencionado	LGPD/GDPR compliance tools + data residency
AI agents	MCP server	MCP + webhooks para agents + batch AI posting
Bulk operations	CSV apenas	CSV + JSON + API batch + webhook-triggered
Real-time	Webhooks apenas	WebSocket + SSE + webhooks (3 opcoes)
Self-host	Nao	Docker compose + Coolify para self-host opcional

12. PLANO DE EXECUCAO (roadmap)

12.1 Fase 1 - Fundacao (semanas 1-3)

1. Setup do projeto: Node.js + NestJS + Fastify + Prisma + PostgreSQL + Redis
2. Schema do banco (tabelas acima) + migracoes
3. Sistema de auth: API keys com hash bcrypt, middleware de validacao
4. CRUD basico: organizations, teams, api_keys
5. Rate limiting middleware (3 camadas com Redis)
6. Health endpoint (/api/v1)

12.2 Fase 2 - Plataformas iniciais (semanas 4-7)

1. Interface PlatformAdapter (padrao Strategy)
2. Implementar 3 plataformas iniciais: Instagram, Facebook, X/Twitter
3. Fluxo OAuth hosted (connect-link) + custom UI
4. Channel selection (set-channel para Meta)
5. Publisher worker (BullMQ) - publica em paralelo por plataforma
6. Sistema de erros padronizado (code, userFacingMessage, isTransient)

12.3 Fase 3 - Midia e publishing completo (semanas 8-10)

1. Upload simple (multipart/form-data)
2. Upload direct (presigned URL)
3. Upload multipart (chunks de 64MiB, retryable)
4. Upload from-url (fetch server-side)
5. Adicionar mais 4 plataformas: TikTok, YouTube, LinkedIn, Threads
6. Scheduler de posts (agenda para postDate)

12.4 Fase 4 - Analytics e webhooks (semanas 11-13)

1. Analytics adapter por plataforma (metrics normalizadas)
2. Raw analytics (payload bruto)
3. Force refresh (rate limited por formula teams x 5)
4. Cron de refresh automatico (24h)
5. Sistema de webhooks: cadastro, signing secret, HMAC-SHA256
6. Webhook delivery worker (3 retries, backoff exponencial, auto-disable)
7. Eventos: post.published, comment.published, social-account.*, team.*

12.5 Fase 5 - Plataformas restantes + extras (semanas 14-18)

1. Adicionar 8 plataformas: Pinterest, Reddit, Discord, Slack, Mastodon, Bluesky, Google Business, Snapchat
2. Comments API (importar e criar comentarios)
3. Post history import (async job)
4. Bulk post from CSV
5. Link in bio
6. Calendar view no dashboard

12.6 Fase 6 - Dashboard + SDK + docs (semanas 19-22)

1. Dashboard Next.js 15: organizacao, teams, contas, posts, calendar, analytics
2. Dark mode + multi-idioma (PT-BR, EN, ES)

3. SDK TypeScript (gerar de OpenAPI com openapi-ts)
4. SDK Python + Go
5. CLI (JSON in/out)
6. MCP server (Claude/Cursor)
7. Docs com Mintlify + llms.txt + OpenAPI Swagger UI

12.7 Fase 7 - Deploy e operacao (semanas 23-24)

1. Docker compose para self-host
2. Deploy em VPS (Coolify) ou Kubernetes
3. Cloudflare na frente (CDN + WAF + SSL)
4. Storage: Cloudflare R2 (zero egress)
5. Monitoring: Grafana + Loki + Prometheus
6. Status page publico
7. Backup automatico do PostgreSQL
8. Testes E2E + load testing

13. INFRAESTRUTURA E HOST

13.1 Custo estimado mensal (self-host)

Componente	Opcao	Custo/mes
VPS (backend + workers)	Hetzner CX32 (8 vCPU, 16GB)	~\$15
VPS (PostgreSQL)	Hetzner CX22 (4 vCPU, 8GB)	~\$10
Redis	Upstash free tier ou self-host na VPS	\$0-10
Storage midia	Cloudflare R2 (10GB free + \$0.015/GB)	~\$0-15
CDN/DNS	Cloudflare free	\$0
Status page	BetterStack free ou Uptime Kuma self-host	\$0
Email (transacional)	Resend free (3000/mes)	\$0
Dominio	.com ou .social	~\$1-3
Monitoring	Grafana self-host	\$0
Backup	B2 Backblaze (\$0.006/GB)	~\$2-5
TOTAL		~\$28-58/mes

13.2 Requisitos para cada plataforma (API keys necessarias)

Plataforma	Onde registrar	Custo
Facebook/Instagram	developers.facebook.com (Meta App)	Gratis (review p/ producao)
X/Twitter	developer.x.com	Free tier limitado; pago p/ volume
TikTok	developers.tiktok.com	Gratis
YouTube	Google Cloud Console (YouTube API)	Gratis (10.000 units/dia)
LinkedIn	developer.linkedin.com	Gratis (review p/ Marketing API)
Threads	developers.threads.com	Gratis
Pinterest	developers.pinterest.com	Gratis
Reddit	reddit.com/prefs/apps	Gratis
Discord	discord.com/developers	Gratis (webhook)
Slack	api.slack.com/apps	Gratis (webhook)
Mastodon	Cada instancia (ou self-host)	Gratis
Bluesky	bsky.social/settings/app-password	Gratis
Google Business	Google Cloud Console (GMB API)	Gratis
Snapchat	snapkit.com	Gratis (review)

14. CHECKLIST DE IMPLEMENTACAO

#	Item	Status
1	Definir stack: NestJS + Fastify + Prisma + PG + Redis	[]
2	Criar schema do banco (orgs, teams, keys, accounts, posts, uploads, webhooks)	[]
3	Sistema de API keys (hash bcrypt, validacao, rotacao)	[]
4	Rate limiting 3 camadas (Redis: 1s/100, 10s/500, 60s/2000)	[]
5	Interface PlatformAdapter + 3 adapters iniciais (IG, FB, X)	[]
6	Fluxo OAuth hosted (connect-link) + callback handling	[]
7	Channel selection (set-channel para Meta platforms)	[]
8	Upload: simple + direct (presigned) + multipart (64MiB chunks)	[]
9	Publisher worker (BullMQ) - publica em paralelo por plataforma	[]
10	Sistema de erros padronizado (code, userFacingMessage, isTransient)	[]
11	Scheduler de posts (agenda para postDate)	[]
12	Webhooks: cadastro, HMAC-SHA256, delivery worker (3 retries)	[]
13	Analytics: adapter por plataforma + refresh 24h + force (rate limited)	[]
14	Comments API (importar + criar)	[]
15	Post history import (async job)	[]
16	Bulk post from CSV	[]
17	Adicionar 12 plataformas restantes	[]
18	Dashboard Next.js 15 (org, teams, contas, posts, calendar, analytics)	[]
19	Dark mode + multi-idioma (PT-BR, EN, ES)	[]
20	SDK TypeScript (openapi-ts) + Python + Go	[]
21	CLI (JSON in/out) + MCP server	[]
22	Docs (Mintlify + llms.txt + Swagger UI)	[]
23	Cron jobs (analytics refresh, token refresh, cleanup, purge)	[]
24	Docker compose + deploy (Coolify ou K8s)	[]
25	Cloudflare (CDN + WAF + SSL) + R2 (storage)	[]
26	Monitoring (Grafana + Loki + Prometheus) + status page	[]
27	Backup automatico PG + testes E2E + load testing	[]
28	Registrar apps em todas as 15 plataformas (API keys)	[]

Nota final: Este documento e uma analise tecnica baseada em documentacao publica, OpenAPI spec, SDK open-source e inferencias arquiteturais. O codigo backend de bundle.social e privado e nao foi acessado. As tecnologias marcadas como '(inferido)' sao estimativas baseadas em evidencias indiretas. Para construir a alternativa, use este blueprint como ponto de partida e adapte conforme sua infraestrutura e necessidades.